

CommonGround — System Architecture

Version: 1.0
Project: Hack4Change 2026 — GMHSC Donation Coordination Platform
Generated: June 2026

Table of Contents

- 1. [Executive Summary](#)
- 2. [High-Level Architecture](#)
- 3. [Technology Stack](#)
- 4. [Project Structure](#)
- 5. [User Roles & Entry Points](#)
- 6. [Backend Architecture](#)
- 7. [Frontend Architecture](#)
- 8. [API Surface](#)
- 9. [Authentication & Security](#)
- 10. [Real-Time Layer \(Socket.IO\)](#)
- 11. [Donation Matching Engine](#)
- 12. [Database Overview](#)
- 13. [How Items Are Stored](#)
- 14. [Domain Workflows](#)
- 15. [Deployment & Operations](#)
- 16. [Testing & Scripts](#)

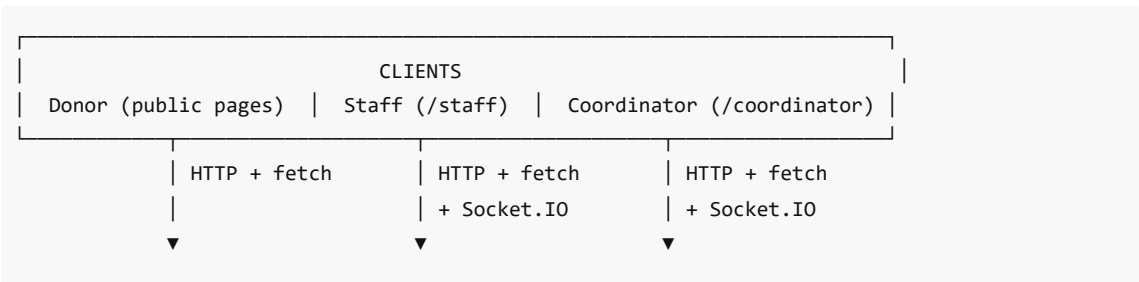
1. Executive Summary

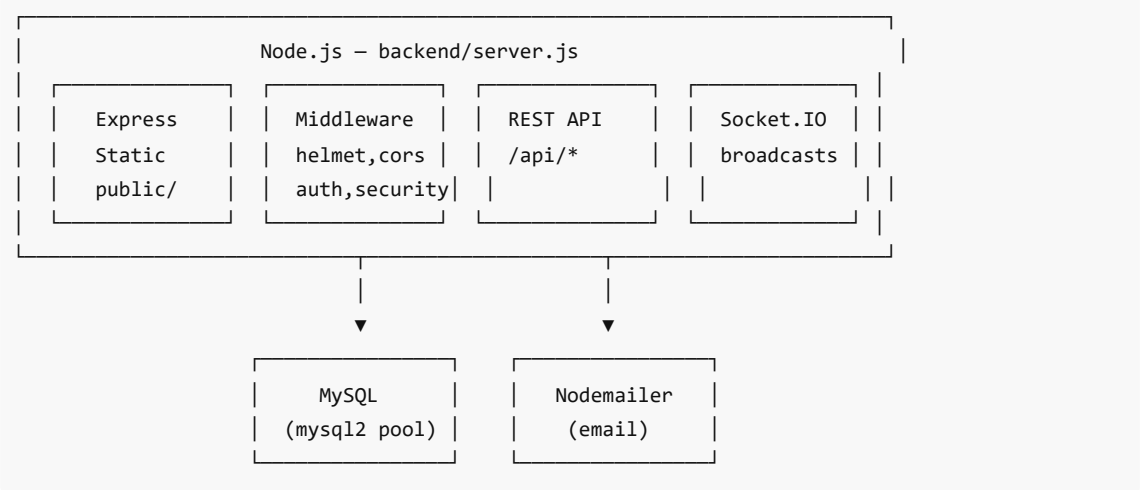
CommonGround is a real-time donation coordination platform for the Greater Moncton Homelessness Steering Committee (GMHSC) network of 28 homeless-serving organizations. It connects three user types in a single web application:

- **Donors** — Browse live needs and submit donations without creating an account.
- **Shelter staff** — Manage org inventory, post shortages, confirm donations, request surplus, and chat.
- **Network coordinators** — View network-wide KPIs, manage organizations and staff, approve transfers, and export reports.

The application runs as a **monolithic Node.js process**: Express serves static HTML/JS from `public/` and a REST API under `/api`, backed by **MySQL**, with **Socket.IO** for live dashboard updates. There is no frontend build step, no microservices layer, and no ORM.

2. High-Level Architecture





- Request flow:**
- 1. Security stack — Helmet CSP, CORS, rate limiting (production), input sanitization, SQL-injection blocking.
 - 2. Route mounting — `/api/auth` , `/api/public` , `/api/staff` , `/api/coordinator` , `/api/ai` .
 - 3. Static files and SPA page routes from `public/` .
 - 4. 404 handling — API returns JSON; unknown pages fall back to `index.html` .

3. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, vanilla JavaScript (no bundler)
Backend	Node.js 18+, Express.js
Database	MySQL via mysql2 connection pool
Real-time	Socket.IO 4.x
Auth	JWT (access + refresh) in httpOnly cookies
Email	Nodemailer
Matching	Rule-based weighted scoring (no external AI API)
Security	Helmet, express-rate-limit, express-validator, bcryptjs
Tests	Jest + Supertest
Deploy	Render (see <code>render.yaml</code>), Aiven MySQL with SSL

4. Project Structure

```
commonGround/
├─ backend/
│   └─ server.js           # Entry: Express + Socket.IO + static routes
│   └─ config/database.js  # mysql2 pool (SSL for cloud)
```

```

|   ├── controllers/           # auth.controller.js
|   ├── middleware/
|   |   ├── auth.middleware.js  # JWT verify, requireAdmin
|   |   └── security.middleware.js
|   ├── models/                # One model per table (parameterized SQL)
|   ├── routes/
|   |   ├── public.routes.js    # No auth
|   |   ├── auth.routes.js
|   |   ├── staff.routes.js     # Org-scoped
|   |   ├── coordinator.routes.js # Network admin
|   |   └── ai.routes.js        # Matching + insights
|   └── utils/                  # email, jwt, hash, logger
└── public/                     # Frontend – served at /
    ├── index.html, donate.html, give.html, all-needs.html
    ├── auth.html, staff.html, coordinator.html
    ├── css/, js/               # api.js, i18n.js, utils.js
    └── admin/                  # Legacy admin HTML (redirects to coordinator)
└── scripts/
    ├── migrate.js              # Destructive schema reset
    └── run-seed-demo.js        # Demo data loader
└── tests/                      # Jest test suites
└── seed-demo.sql               # Moncton org demo data
└── render.yaml                 # Render deployment blueprint
└── package.json
└── .env.example

```

5. User Roles & Entry Points

User Type	Entry Point	Primary URL	Auth
Donor	Homepage, Give, All Needs	/, /give, /donate, /all-needs	None
Staff	Login	/login → /staff	JWT (org-scoped)
Coordinator	Login	/login → /coordinator	JWT + isAdmin

Public pages

URL	Description
/	Live needs board — all orgs with top needs
/give	Donation landing — money vs items
/donate	4-step donation form with auto-matching
/all-needs	Paginated network needs list
/login	Staff and coordinator login

Staff dashboard sections

Dashboard, Inventory, Your Needs, Other Needs, Donations, Analytics, Chats.

Coordinator dashboard sections

Overview, Organizations, Staff, All Needs, Donations, Inventory, Transfers, Chat, Analytics.

6. Backend Architecture

6.1 Layering

Layer	Location	Responsibility
Routes	backend/routes/	HTTP handlers, validation, emit Socket events
Models	backend/models/	Parameterized SQL queries per table
Controllers	backend/controllers/	Auth logic (login, refresh, cookies)
Middleware	backend/middleware/	JWT, admin gate, sanitization
Utils	backend/utils/	Cross-cutting helpers

6.2 Models (data access)

Model	Table(s)
organization.model.js	organizations
staff.model.js	staff_members
admin.model.js	admins
inventory.model.js	inventory_items
needs.model.js	needs
donation.model.js	donations
surplusRequest.model.js	surplus_requests
surplusTransfer.model.js	surplus_transfers
chatThread.model.js	chat_threads
chatMessage.model.js	chat_messages

6.3 Route authorization

/api/public/*	→ No authentication
/api/auth/*	→ Public (login) + cookie-based session
/api/staff/*	→ authenticateToken (staff JWT)
/api/coordinator/*	→ authenticateToken + requireAdmin
/api/ai/match-donation	→ Public (rate-limited)
/api/ai/network-insights	→ Admin only

7. Frontend Architecture

Pattern: Multi-page app with two large single-file dashboard SPAs (`staff.html` , `coordinator.html`).

Concern	Implementation
API client	<code>public/js/api.js</code> — fetch with credentials: 'include'
i18n	<code>public/js/i18n.js</code> — EN/FR toggle
Theming	<code>theme-init.js</code> , <code>site-prefs.js</code> , <code>theme.css</code>
Dashboard UX	Sidebar section switching; no full page reloads
Live updates	Socket.IO listeners on staff/coordinator pages

API client modules (`api.js`)

- `auth` — me, login, logout, refresh
 - `pub` — `orgsWithNeeds`, `needs`, `submitDonation`, `applyMatch`
 - `staff` — `inventory`, `needs`, `donations`, `surplus`, `transfers`, `chat`
 - `coordinator` — `overview`, `orgs`, `staff`, `exports`, `analytics`
 - `ai` — `matchDonation`, `networkInsights`
-

8. API Surface

Public (no JWT)

- `GET /api/public/orgs-with-needs` — Needs board data
- `GET /api/public/needs` — Filtered unfulfilled needs
- `GET /api/public/organizations` — Org list for dropdowns
- `POST /api/public/donations` — Submit donation
- `PATCH /api/public/donations/:id/match` — Apply match result

Auth

- `POST /api/auth/login` — Login (pass `role` : staff or coordinator)
- `POST /api/auth/logout`
- `GET /api/auth/me`
- `POST /api/auth/refresh`

Staff (JWT required)

- `GET /api/staff/org`
- `GET|POST /api/staff/inventory` , `PATCH|DELETE /api/staff/inventory/:id`
- `GET|POST /api/staff/needs` , `PATCH|DELETE /api/staff/needs/:id`
- `POST /api/staff/needs/:id/fulfill` , `POST /api/staff/needs/:id/receive`
- `GET /api/staff/donations` , `POST /api/staff/donations/:id/confirm`
- `GET /api/staff/surplus` , `GET|POST /api/staff/surplus-requests`
- `GET /api/staff/transfers` , `POST /api/staff/transfers/:id/complete`
- `GET /api/staff/expiring?days=30`
- Chat: threads, messages, cross-org threads

Coordinator (JWT + admin)

- GET /api/coordinator/overview
- GET|POST|PATCH /api/coordinator/orgs
- GET|POST|PATCH|DELETE /api/coordinator/staff
- GET /api/coordinator/needs , GET /api/coordinator/donations
- PATCH /api/coordinator/donations/:id/status
- GET /api/coordinator/inventory , GET /api/coordinator/surplus
- GET|PATCH /api/coordinator/surplus-requests
- GET|POST|PATCH /api/coordinator/transfers
- GET /api/coordinator/analytics
- CSV exports: needs, inventory, donations, orgs, staff

Matching (/api/ai)

- POST /api/ai/match-donation — Top 3 org matches (rule-based)
- POST /api/ai/network-insights — Network summary (admin only)

Health

- GET /api/health — Basic liveness
- GET /api/health?deep=true — Includes MySQL ping

9. Authentication & Security

Authentication flow

1. User posts email + password + role to /api/auth/login .
2. Server verifies against staff_members or admins table (bcrypt).
3. Access + refresh JWTs set as **httpOnly cookies**.
4. Protected routes read req.cookies.token or Authorization: Bearer .
5. Coordinator tokens include isAdmin: true ; requireAdmin middleware gates coordinator routes.

Security controls

Control	Implementation
HTTP headers	Helmet + custom CSP
CORS	Configurable via CORS_ORIGINS; credentials enabled
Rate limiting	express-rate-limit in production (200 req / 15 min default)
Input sanitization	security.middleware.js
SQL injection	Pattern blocking + parameterized queries in models
Passwords	bcryptjs hashes
Cookies	COOKIE_SECURE, COOKIE_SAME_SITE for production

10. Real-Time Layer (Socket.IO)

Socket.IO is attached to the same HTTP server as Express (app.set('io', io)).

Events emitted from server

Event	Trigger
donation:new	Public donation submitted
need:new	Staff posts a need
need:fulfilled	Need marked fulfilled
chat:newMessage	Message sent in chat thread

Staff and coordinator dashboards subscribe to these events to refresh UI without polling.

11. Donation Matching Engine

Path: POST /api/ai/match-donation

Type: Rule-based scoring — no external LLM or API keys required.

Scoring weights

Factor	Points
Category match (with aliases)	+10
Item name similarity (substring / word overlap)	+15
Urgency (critical=4, high=3, medium=2, low=1) × 3	up to +12
Donor quantity ≥ need quantity	+5

Process

- 1. Load all unfulfilled needs (NeedsModel.findAllUnfulfilled()).
- 2. Score each need against donor item name, category, quantity.
- 3. Filter score > 0, sort descending.
- 4. Return **top 3** organizations with plain-language reasoning.

Network insights (POST /api/ai/network-insights): rule-based summary of critical needs, surplus items, redistribution opportunities, and stale inventory orgs.

12. Database Overview

Entity relationship (core tables)

organizations (1) —< (N) staff_members
organizations (1) —< (N) inventory_items
organizations (1) —< (N) needs
organizations (1) —< (N) donations [preferred_org_id, matched_org_id]
inventory_items (1) —< (N) surplus_requests
inventory_items (1) —< (N) surplus_transfers

```
chat_threads (1) —< (N) chat_messages
organizations — chat_threads (org_channel, cross_org)
staff_members — chat_threads (direct)

admins — network coordinators (not org-scoped)
```

All tables

Table	Purpose
admins	Coordinator accounts
organizations	28 network orgs (shelters, food banks, etc.)
staff_members	Org-scoped login users
inventory_items	Per-org stock on hand
needs	Per-org shortages
donations	Donor submissions + pipeline status
surplus_requests	Staff requests surplus from another org
surplus_transfers	Coordinator-initiated stock moves
chat_threads	org_channel, direct, cross_org
chat_messages	Thread messages

Schema management

- **Create/reset:** `npm run migrate` — drops and recreates all tables (destructive).
- **Seed demo:** `npm run seed:demo` — loads `seed-demo.sql`.
- **Full setup:** `npm run setup` — install + migrate + seed.

Key enums

Organization category: `shelter_housing`, `food_nutrition`, `goods_essentials`, `mental_health`, `outreach`

Inventory status: `available`, `low`, `critical`, `surplus` (computed from quantity vs target_quantity)

Need urgency: `low`, `medium`, `high`, `critical`

Donation status: `pending` → `matched` → `confirmed` → `delivered`

Transfer status: `pending`, `in_transit`, `completed`, `cancelled`

13. How Items Are Stored

No global product catalog

There is **no** shared `items` or `products` table. Every good is stored as **org-owned rows** with free-text `item_name` (e.g. "Shelf-Stable Milk (1L)").

Three representations of an item

Concept	Table	Scoped by	Uniqueness rule
Stock on hand	inventory_items	org_id	One row per (org + item_name + category)
Shortage	needs	org_id	One open need per (org + item_name + category)
Incoming gift	donations	donor + matched org	Standalone until matched

Inventory row fields

- `id` (UUID), `org_id`, `item_name`, `category`, `quantity`, `target_quantity`, `unit`
- `status` — auto-computed: below target = low, above target = surplus
- `expiry_date`, `notes`

Example: all 28 orgs have milk

In inventory: 28 separate rows in `inventory_items`, each with its own UUID, quantity, status, and expiry. No shared pool.

In needs: Up to 28 open need rows. Donation matcher scores all of them and returns **top 3** by urgency and name match; donor picks one `matched_org_id`.

Surplus transfer: References a specific `inventory_item_id` (one org's milk row), not "milk" globally.

After staff confirms donation: `InventoryModel.addQuantityForNeed()` merges quantity into **only** the matched org's row (or creates one).

Status computation (inventory)

```
if target_quantity <= 0: status = quantity > 0 ? 'available' : 'low'
if quantity < target:   status = 'low'
if quantity > target:   status = 'surplus'
else:                   status = 'available'
```

14. Domain Workflows

14.1 Donation lifecycle

```
Donor submits form (/donate)
  → POST /api/public/donations (status: pending)
  → Confirmation email to donor
  → Socket.IO: donation:new
  → POST /api/ai/match-donation (top 3 orgs)
  → PATCH /api/public/donations/:id/match (status: matched)
  → Staff sees in Donations tab
  → POST /api/staff/donations/:id/confirm (status: confirmed)
  → Inventory quantity increased for matched org
```

- Thank-you email to donor
- Coordinator may set status: delivered (resolved)

14.2 Surplus redistribution

Staff A flags item as surplus (quantity > target_quantity)
→ Coordinator sees in surplus inventory view
Staff B posts need for same item (separate need row)
→ Staff B requests surplus OR Coordinator initiates transfer
→ Coordinator approves surplus request (if applicable)
→ Transfer tracked (from_org, to_org, inventory_item_id)
→ Staff B confirms receipt → transfer completed

14.3 Staff need management

Staff posts need → need:new event
Partial fulfillment via receive → quantity_needed decremented
Full fulfillment → fulfilled = 1, fulfilled_at set

15. Deployment & Operations

Local development

```
npm install
cp .env.example .env # configure DB_*, JWT_SECRET
npm run setup         # migrate + seed
npm run dev           # nodemon on port 3000
```

Production (Render)

- **Blueprint:** render.yaml
- **Start:** npm start → node backend/server.js
- **Health check:** /api/health
- **Database:** Aiven MySQL with DB_SSL_ENABLED=true and ca.pem
- **Env:** NODE_ENV=production, COOKIE_SECURE=true, generated JWT_SECRET

Environment variables (essential)

Variable	Purpose
PORT	Server port (default 3000)
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME	MySQL connection
DB_SSL_ENABLED, DB_CA_CERT_PATH	Cloud MySQL SSL
JWT_SECRET	Token signing (required)
CORS_ORIGINS	Allowed origins
ADMIN_DEFAULT_EMAIL, ADMIN_DEFAULT_PASSWORD	Coordinator seed

16. Testing & Scripts

Test suites (`npm test`)

- `api.test.js` , `auth.test.js` , `models.test.js`
- `security.test.js` , `security-audit.test.js`
- `database.test.js` , `hash.test.js` , `logger.test.js` , `i18n.test.js`

NPM scripts

Script	Command
<code>setup</code>	<code>install + migrate + seed:demo</code>
<code>dev</code>	<code>nodemon backend/server.js</code>
<code>start</code>	<code>node backend/server.js</code>
<code>migrate</code>	Reset DB schema
<code>seed:demo</code>	Load demo SQL
<code>test</code>	Jest with <code>forceExit</code>

Appendix: Design Principles

1. **Monolith simplicity** — One process, one deploy unit, no frontend build.
2. **Org-scoped data** — Inventory and needs belong to organizations; coordinators aggregate via queries.
3. **Free-text items** — Flexible for real-world shelter naming; matching uses fuzzy string logic.
4. **Progressive auth** — Donors need no account; staff/coordinators use JWT cookies.
5. **Real-time where it matters** — Donations and needs push to dashboards via Socket.IO.
6. **Rule-based intelligence** — Matching and insights work offline without paid AI APIs.